Programowanie Aplikacji INternetowych

strona vs aplikacja internetowa

strona internetowa to dokument, a aplikacja internetowa to program dostępny w przeglądarce

przegladarka

- program do przeglądania stron i aplikacji internetowych
- komunikuje sie z serwerem - pobiera i wysyła zasoby
- interpretuje HTML, CSS i JavaScript
- umożliwia wykonywanie skryptów
- przykłady: Chrome, Firefox, Edge, Safari
- DevTools - narzędzia dla programisty

skrypt

- program wykonywany w określonym środowisku
- zwykle nie wymaga klasycznego procesu kompilacji przed uruchomieniem
- może działać po stronie klienta lub serwera
- skrypt może m.in. przetwarzać dane, reagować na działania użytkownika, komunikować się z serwerem

środowisko uruchomieniowe

- środowisko umożliwiające wykonywanie programu/skryptu
- dostarcza potrzebnych mechanizmów i bibliotek
- określa, w jakich warunkach działa kod
- zapewnia dostęp do wybranych zasobów systemu

napisz swój pierwszy skrypt

Utwórz plik index.html z podstawową strukturą strony

1. <script> console.log("Witaj szkoło!"); </script>
2. <script src="skrypt.js"></script>

co się dzieje w przeglądarce?

dynamiczne typowanie

```
let x = “10”
console.log(x+5);

console.log(x == 10);
console.log(x === 10);
```

let, const, var

```
if (true) {
    let x = 10;
}

console.log(x);
```

```
if (true) {
    var x = 10;
}

console.log(x);
```

typy danych

```
let age = 18;
let price = 19.99;
let name = "Milena";
let isStudent = false;
let value = null;
let x;

console.log(typeof age);
```

konwersja typów - cast

```
let text = "123";
let number = Number(text);
let text2 = number.toString();

console.log(number);
console.log(typeof number);
```

funkcje

```
function hello(name) {
    console.log("Hello " + name);
}

const function = function(name) { console.log("Hello " +
name); };

hello("Milena");
```

funkcje strzałkowe

```
const dodawanie = (a, b) => {
    return a + b;
};

const dodawanie = (a, b) => a + b;
```

zadanie!

Zadeklaruj zmienne:
let username = "Kuba";
let age = "17";
let password = "abc12345";

Napisz funkcję isValidUser(username, age, password)która zwraca true jeżeli:

- nazwa użytkownika ma co najmniej 3 znaki,
- użytkownik ma co najmniej 13 lat,
- hasło ma co najmniej 8 znaków.

W przeciwnym wypadku zwraca false. Przetestuj funkcję dla zadeklarowanych zmiennych.

zadanie!

Napisz funkcję w języku JavaScript, która dla zadeklarowanych zmiennych stałych **month** i **year** zwróci liczbę dni danego miesiąca. Jeśli podanym miesiącem jest luty, funkcja musi sprawdzić, czy podany rok jest przestępny. Rok jest przestępny, jeśli jest podzielny przez 4 i nie jest podzielny przez 100, LUB jest podzielny przez 400. Zwrócony wynik działania funkcji wypisz w konsoli w postaci zdania np. “Luty 2024 ma 29 dni.”

zadanie!

Dana jest tablica, w której każdy element to podany wiek osoby z grupy, która kupuje bilety do ZOO:

const **daneWejsciowe** = [2, "12", "30", "67", "tekst", null, 18];

Napisz funkcję, która dla każdego elementu tablicy obliczy cenę biletu na podstawie poniższych reguł.

- >3 lat wejście darmowe
- 3-18 lat bilet ulgowy 15 zł
- 65+ lat bilet seniora 20 zł
- bilet normalny 30 zł

operacje na tablicach - map

Tworzy nową tablicę poprzez wykonanie funkcji na każdym elemencie tablicy. Argumentem jest funkcja (nie wywołanie funkcji!)

```
const numbers = [4, 9, 16, 25];
const newArr = numbers.map(Math.sqrt)
```

operacje na tablicach - filter

Tworzy nową tablicę ze wszystkimi elementami tablicy wejściowej, które przejdą test.

```
const ages = [32, 33, 16, 40];
const result = ages.filter(checkAdult);

function checkAdult(age) {
  return age >= 18;
}
```

operacje na tablicach - find

Zwraca wartość pierwszego elementu, który przejdzie test.
W przypadku nie znalezienia zwraca undefined.

```
const ages = [3, 10, 18, 20];

function checkAge(age) {
  return age > 18;
}

ages.find(checkAge);
```

zadanie!

```
const prices = [12, 45, 8, 120, 35, 200, 17, 60];
```

1. Utwórz nową tablicę zawierającą ceny wszystkich produktów powiększone o 23% podatku VAT.
2. Utwórz nową tablicę zawierającą wyłącznie ceny produktów droższych niż 50 zł.
3. Znajdź pierwszą cenę w tablicy, która jest większa niż 100 zł
4. Znajdź pierwszą cenę w tablicy, która jest większa niż 1000 zł. Jeśli program nie znajdzie odpowiedniej ceny, wyświetl komunikat „Nie znaleziono produktu droższego niż 1000 zł”.

obiekty w javascript

Obiekt to zbiór powiązanych ze sobą danych:

```
const uczen = {
    name: "Kasia",
    class: "4b",
    grades: [5, 4, 3, 5]
};
```

properties

odczytywanie wartości:
console.log(student.name);
console.log(student[“class”]);

odczytywanie wartości poprzez zmienną:
const propertyName = “grades”;
console.log(student[propertyName]);

modyfikacja wartości:
student.name = “Basia”;
student.school = “SCI”;

metody obiektów

```
const product = {
    name: "Monitor",
    price: 900,
    applyDiscount(percent) {
        this.price = this.price * (1 - percent / 100);
    }
};

console.log(product.price);
product.applyDiscount(10);
console.log(product.price);
```

zagnieżdżone obiekty

```
const order = {
    number: 2137,
    customer: {
        name: "Milena",
        city: "Szczecin"
    },
    products: [
        { name: "Mysz", price: 80, quantity: 2 },
        { name: "Monitor", price: 900, quantity: 1 }
    ]
};

console.log(order.customer.name);
console.log(order.products[0].price);
```

zadanie!

Stwórz tablicę **products** będącą zbiorem produktów pewnego sklepu internetowego. Każdy produkt ma swoje id, nazwę, cenę, ilość danych produktów w magazynie i minimalną ilość. Dodaj minimum 3 różne produkty do tablicy.

1. Oblicz i zwróć wartość wszystkich produktów z tablicy (suma cena * ilość).
2. Stwórz wyświetlony w konsoli raport o niskich stanach magazynowych, który wyświetli wszystkie produkty których ilość w magazynie jest mniejsza niż minimalna ilość. Zadbaj o odpowiednia informację dla użytkownika.

porównywanie obiektów

```
const a = { price: 100 };
const b = { price: 100 };
const c = a;

console.log(a === b); // false
console.log(a === c); // true
```

kopiowanie obiektów

```
const original = {
    name: "Laptop",
    price: 3000
};

const copy = { ...original };

copy.price = 2500;

console.log(original.price); // 3000
console.log(copy.price);     // 2500
```

płytka vs głęboka kopia

Spread (...) tworzy płytką (shallow) kopię obiektu. Jeśli obiekt zawiera zagnieżdżone obiekty lub tablice, w kopii przechowywane są jedynie referencje do nich (czyli wskazują na te same miejsca w pamięci co oryginał).

Głęboką (deep) kopię można uzyskać np. poprzez zamianę na obiekt JSON:
let cloneUser = JSON.parse(JSON.stringify(userDetails));

przekazywanie obiektów

Obiekt przekazany do funkcji może zostać zmodyfikowany przez tę funkcję.

```
const product = { name: "Laptop", price: 3000};

function calculateVATPrice(product) {
    return product.price * (1 + 23 / 100);
}
```

zadanie!

Napisz funkcje lub metody do operacji na tablicy **products** z poprzedniego zadania.

1. findProductById - zwraca produkt o podanym identyfikatorze (jeśli nie znajdzie - undefined).
2. searchProducts(string) - wyszukuje produkty, których nazwa zawiera podany fragment tekstu, niezależnie od wielkości liter np. argument “mysz” zwraca produkt “Mysz gamingowa”.
3. addProduct(id, name, price, quantity, minimum) - która dodaje nowy produkt do tablicy (*array push*)

try, catch

Jeśli chcemy przeprowadzić operację, która może się nie udać, możemy użyć try.

```
try {
    console.log("Wykonuję operację");
    throw new Error("Awaria");
} catch (error) {
    console.log(error.message);
}
```

finally

Blok finally wykonuje się po try i catch niezaleznie czy zostały wyłapane jakieś błędy.

```
try {
    console.log("Wykonuję operację");
    throw new Error("Awaria");
} catch (error) {
    console.log(error.message);
} finally {
    console.log("Koniec operacji");
}
```

zadanie!

Dodaj do funkcji sellProduct wyłapywanie błędnie podanych danych wejściowych, np. sellProduct(10, 2, 44) lub produktu którego id nie zostało znalezione.

funkcja generator, yield

```
function* gen(index) {
  while (index < 2) {
    yield index;
    index++;
  }
}

const iterator = gen(0);

console.log(iterator.next().value);
console.log(iterator.next().value);
```

definiowanie własnych obiektów

Deklaracja klasy:

```
class Rectangle {
    constructor(height, width) {
        this.height = height;
        this.width = width;
    }
}
```

```
class Rectangle {
    constructor(height, width) {
        this.height = height;
        this.width = width;
    }
    get area() {
        return this.calcArea();
    }
    calcArea() {
        return this.height * this.width;
    }
    *getSides() {
        yield this.height;
        yield this.width;
        yield this.height;
        yield this.width;}
}
```

getter

metoda

generator

```
const square = new Rectangle(10, 10);

console.log(square.area);
console.log([...square.getSides()]);
```

zadanie!

Zdefiniuj klasę **Task**, która reprezentuje pojedyncze zadanie do wykonania i posiada **tytuł, priorytet**(1-3) i flagę **isCompleted**. Konstruktor przyjmuje nazwę i priorytet. Początkowo zadanie nie jest wykonane. Stwórz następujące metody:

- **complete**() - oznacza zadanie jako wykonane
- **changePriority**(newPriority) - zmienia priorytet na podany
- **getInfo**() - zwraca tekst zawierający nazwę, priorytet oraz status zadania

Test nowej klasy:

```
const task1 = new Task("Przygotować projekt", 3);
const task2 = new Task("Nauczyć się JavaScriptu", 2);

console.log(task1.getInfo());
console.log(task2.getInfo());

task1.complete();
task2.changePriority(1);

console.log(task1.getInfo());
console.log(task2.getInfo());
```